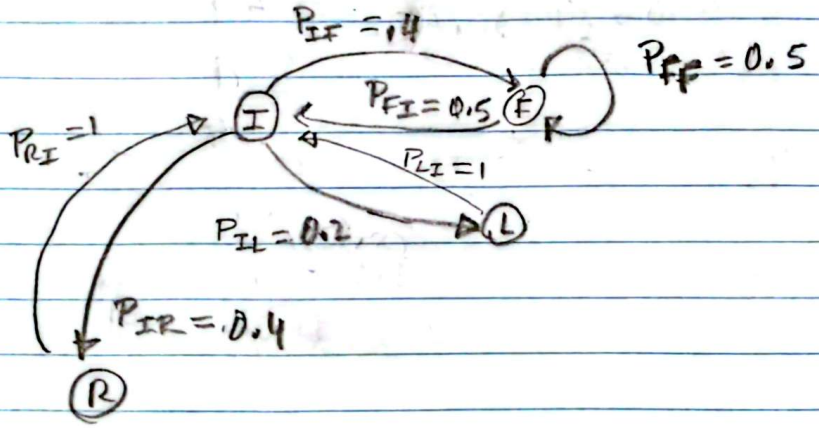
ASE 379L HW5

1.

1.1 $S: \{I, F, L, R\}$

I: at intersection → stops
F: forward
L: left
R: right

1.2



$P_{FI} = P_{LI} = P_{RI} = 1$ since the car always goes to the next intersection I → this is a deterministic step

$P_{IF} = .4$

$P_{IL} = 0.2$

$P_{IR} = 0.4$

$P_{IL} = 0.2$

$P_{FF} = 0.5$ (if it does not stop, we do not turn/ move to another state).

$P_{FI} = 0.5$ ↕ equal probability between stopping and continuing forward

1.3 Transition kernal P:

let state $= \begin{Bmatrix} I \\ F \\ L \\ R \end{Bmatrix} = \vec{u}$

then: $P = \begin{bmatrix} 0 & 0.4 & 0.2 & 0.4 \\ 0.5 & 0.5 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$

(rows sum to 1)

1.4 $P_I = P_F = P_L = P_R$ } at $t=0$.
$= 0.25$.

So, $\boxed{p_0 = \begin{bmatrix} .25 \\ .25 \\ .25 \\ .25 \end{bmatrix}}$

at step 10, we have $(P)^{10} p_0$.

$p_{10} = (P)^{10} p_0 \approx \begin{bmatrix} 0.52 & 0.30 & 0.0589 & 0.17 \\ 0.38 & 0.34 & 0.092 & 0.18 \\ 0.29 & 0.37 & 0.112 & 0.22 \\ 0.29 & 0.37 & 0.112 & 0.22 \end{bmatrix}$

$= [0.37, 0.346, 0.09, 0.19]$

2. For a diagonal transition matrix, transitions between states cannot occur.

Each row must sum to 1, since, from a given state, the probability of moving to any other possible state (including remaining at the current state) must sum to 100%.
As such $A_i = 1 \quad \forall i \in [1, N]$

So, $P = I_N$ (identity of size $N \times N$)

and application of P on any state vector

p_0 will cause $p_1 = p_0 P = p_0$.
↑
Step 0

The chain is thus reducible, with n independent states

p1

Exported 2/14/2026, 5:13:31 PM

In [7]:

```
import numpy as np
P = np.array([[0,0.4,0.2,0.4],[0.5, 0.5,0,0],[1,0,0,0],[1,0,0,0]])
p0 = np.ones(4)/4
p10 = p0 @ np.linalg.matrix_power(P, 10)
print("P**10", np.linalg.matrix_power(P, 10))
p10
```

```
P**10 [[0.52088625 0.30249625 0.0588725  0.117745  ]
 [0.37812031 0.34475281 0.09237563 0.18475125]
 [0.2943625  0.3695025  0.112045   0.22409   ]
 [0.2943625  0.3695025  0.112045   0.22409   ]]
```

```
array([0.37193289, 0.34656352, 0.09383453, 0.18766906])
```

1.5

In [13]:

```
np.random.seed(42)
def simulate_chain(P, p0, N):
    states = np.zeros(N, dtype=int)
    states[0] = np.random.choice(4, p=p0)

    for n in range(1, N):
        states[n] = np.random.choice(4, p=P[states[n-1]])

    return states
def empirical_distribution(states, n):
    counts = np.bincount(states[:n], minlength=4)
    return counts / n
N = 100
print(f"N={N}")
states = simulate_chain(P, p0, N)

# empirical distributions at even and odd times
p_even = empirical_distribution(states[::2], N // 2)
p_odd  = empirical_distribution(states[1::2], N // 2)

print("Empirical distribution (even times):", p_even)
print("Empirical distribution (odd times): ", p_odd)

N = 1000000
print(f"N={N}")
states = simulate_chain(P, p0, N)

# empirical distributions at even and odd times
p_even = empirical_distribution(states[::2], N // 2)
p_odd  = empirical_distribution(states[1::2], N // 2)

print("Empirical distribution (even times):", p_even)
print("Empirical distribution (odd times): ", p_odd)
```

```
N=100
Empirical distribution (even times): [0.48 0.46 0.04 0.02]
Empirical distribution (odd times):  [0.24 0.56 0.04 0.16]
N=1000000
Empirical distribution (even times): [0.417396 0.332328 0.083168
0.167108]
Empirical distribution (odd times):  [0.416444 0.332076 0.0837
0.16778 ]
```

The distribution appears to converge to a stationary distribution with approximate probability 0.41 of being at an intersection, probability 0.33 of going forward, 0.083 of turning left and 0.17 of turning right. We expect this, since we have an irreducible chain (every state can reach every other state), finite number of states, and aperiodicity (the states have a self-loop: they can return to themslves in 1 step).

p3

Exported 2/14/2026, 5:13:45 PM

3

In [10]:

```
import numpy as np

P = np.array([
    [0, 1, 0, 0, 0],
    [0, 0, 1, 0, 0],
    [0.5, 0, 0, 0.5, 0],
    [0, 0, 0, 0, 1],
    [0, 0, 1, 0, 0]
])

pi = np.array([1/6, 1/6, 1/3, 1/6, 1/6])

print("pi P =", pi @ P)
print("pi   =", pi)
```

```
pi P = [0.16666667 0.16666667 0.33333333 0.16666667 0.16666667]
pi   = [0.16666667 0.16666667 0.33333333 0.16666667 0.16666667]
```

Since $\pi P = \pi$, we have a stationary distribution.

In [11]:

```
p0 = np.array([0.5, 0, 0, 0.5, 0])

def evolve(p, P, n):
    for _ in range(n):
        p = p @ P
    return p
# p0 = np.array([1,0,0,0,0])
for n in [3*n for n in range(0, 12)]:
    print(f"p_{n} =", evolve(p0, P, n))
```

```
p_0 = [0.5 0.  0.  0.5 0. ]
p_3 = [0.5 0.  0.  0.5 0. ]
p_6 = [0.5 0.  0.  0.5 0. ]
p_9 = [0.5 0.  0.  0.5 0. ]
p_12 = [0.5 0.  0.  0.5 0. ]
p_15 = [0.5 0.  0.  0.5 0. ]
p_18 = [0.5 0.  0.  0.5 0. ]
p_21 = [0.5 0.  0.  0.5 0. ]
p_24 = [0.5 0.  0.  0.5 0. ]
p_27 = [0.5 0.  0.  0.5 0. ]
p_30 = [0.5 0.  0.  0.5 0. ]
p_33 = [0.5 0.  0.  0.5 0. ]
```

Since $p_0 = p_3 = \ldots . p_{3n}$, the Markov chain is periodic with period $k = 3$.

p4

Exported 2/14/2026, 5:13:54 PM

4

In [2]:

```
import numpy as np

P = np.array([
    [0, 1, 0, 0],
    [1/3, 0, 2/3, 0],
    [0, 0, 0, 1],
    [1/2, 0, 1/2, 0]
])

def evolve(p, P, n):
    for _ in range(n):
        p = p @ P
    return p
```

In [4]:

```
p0 = np.array([1/4, 1/4, 1/4, 1/4])

for n in range(0, 20):
    print(f"p_{n} =", evolve(p0, P, n))
```

```
p_0 = [0.25 0.25 0.25 0.25]
p_1 = [0.20833333 0.25       0.29166667 0.25      ]
p_2 = [0.20833333 0.20833333 0.29166667 0.29166667]
p_3 = [0.21527778 0.20833333 0.28472222 0.29166667]
p_4 = [0.21527778 0.21527778 0.28472222 0.28472222]
p_5 = [0.21412037 0.21527778 0.28587963 0.28472222]
p_6 = [0.21412037 0.21412037 0.28587963 0.28587963]
p_7 = [0.21431327 0.21412037 0.28568673 0.28587963]
p_8 = [0.21431327 0.21431327 0.28568673 0.28568673]
p_9 = [0.21428112 0.21431327 0.28571888 0.28568673]
p_10 = [0.21428112 0.21428112 0.28571888 0.28571888]
p_11 = [0.21428648 0.21428112 0.28571352 0.28571888]
p_12 = [0.21428648 0.21428648 0.28571352 0.28571352]
p_13 = [0.21428559 0.21428648 0.28571441 0.28571352]
p_14 = [0.21428559 0.21428559 0.28571441 0.28571441]
p_15 = [0.21428574 0.21428559 0.28571426 0.28571441]
p_16 = [0.21428574 0.21428574 0.28571426 0.28571426]
p_17 = [0.21428571 0.21428574 0.28571429 0.28571426]
p_18 = [0.21428571 0.21428571 0.28571429 0.28571429]
p_19 = [0.21428571 0.21428571 0.28571429 0.28571429]
```

Yes, when initialized at $p_0 = [1 / 4, 1 / 4, 1 / 4, 1 / 4]$, the chain appears to convege near $p_n = [.21428571, .21428571, 0.28571429, 0.28571429]$.

In [8]:

```
p0 = np.array([1, 0, 0, 0])

for n in range(0, 20):
    print(f"p_{n} =", evolve(p0, P, n))
```

```
p_0 = [1 0 0 0]
p_1 = [0. 1. 0. 0.]
p_2 = [0.33333333 0.         0.66666667 0.        ]
p_3 = [0.         0.33333333 0.         0.66666667]
p_4 = [0.44444444 0.         0.55555556 0.        ]
p_5 = [0.         0.44444444 0.         0.55555556]
p_6 = [0.42592593 0.         0.57407407 0.        ]
p_7 = [0.         0.42592593 0.         0.57407407]
p_8 = [0.42901235 0.         0.57098765 0.        ]
p_9 = [0.         0.42901235 0.         0.57098765]
p_10 = [0.42849794 0.         0.57150206 0.        ]
p_11 = [0.         0.42849794 0.         0.57150206]
p_12 = [0.42858368 0.         0.57141632 0.        ]
p_13 = [0.         0.42858368 0.         0.57141632]
p_14 = [0.42856939 0.         0.57143061 0.        ]
p_15 = [0.         0.42856939 0.         0.57143061]
p_16 = [0.42857177 0.         0.57142823 0.        ]
p_17 = [0.         0.42857177 0.         0.57142823]
p_18 = [0.42857137 0.         0.57142863 0.        ]
p_19 = [0.         0.42857137 0.         0.57142863]
```

There seems to be an oscillation between two modes (even/odd steps), especially at later steps, where all the mass is on either the even indices or the odd indices. The chain seems to be converging to an oscillatory sequence. To confirm, we check larger time steps of the chain:

In [9]:

```
for n in range(400, 420):
    print(f"p_{n} =", evolve(p0, P, n))
```

```
p_400 = [0.42857143 0.         0.57142857 0.        ]
p_401 = [0.         0.42857143 0.         0.57142857]
p_402 = [0.42857143 0.         0.57142857 0.        ]
p_403 = [0.         0.42857143 0.         0.57142857]
p_404 = [0.42857143 0.         0.57142857 0.        ]
p_405 = [0.         0.42857143 0.         0.57142857]
p_406 = [0.42857143 0.         0.57142857 0.        ]
p_407 = [0.         0.42857143 0.         0.57142857]
p_408 = [0.42857143 0.         0.57142857 0.        ]
p_409 = [0.         0.42857143 0.         0.57142857]
p_410 = [0.42857143 0.         0.57142857 0.        ]
p_411 = [0.         0.42857143 0.         0.57142857]
p_412 = [0.42857143 0.         0.57142857 0.        ]
p_413 = [0.         0.42857143 0.         0.57142857]
p_414 = [0.42857143 0.         0.57142857 0.        ]
p_415 = [0.         0.42857143 0.         0.57142857]
p_416 = [0.42857143 0.         0.57142857 0.        ]
p_417 = [0.         0.42857143 0.         0.57142857]
p_418 = [0.42857143 0.         0.57142857 0.        ]
p_419 = [0.         0.42857143 0.         0.57142857]
```

Clearly, the distribution converges to an oscllatory distribution, where it is either on an even mode/step (then, with distribution of [0.42857143, 0. , 0.57142857 0.,]) or an odd mode (with distribution of [0., 0.42857143, 0. ,0.57142857]). Note that the probability distribution is discrete (e.g., only mass on the 1st/3rd states or the 2nd/4th states), and the transition matrix shifts the mass either to the right or left.

We do not get the same limit as with an initial distribution of $p_0 = [1 / 4, 1 / 4, 1 / 4, 1 / 4]$, which converges to a stationary point. Instead we get a distribution with a period of 2.

p5

Exported 2/14/2026, 5:14:02 PM

5

In [2]:

```
import numpy as np

P = np.array([
    [1/3, 2/3],
    [1,   0  ]
])

for m in range(1, 6):
    Pm = np.linalg.matrix_power(P, m)
    print(f"P^{m} =\n{Pm}\n")
for m in range(2, 600):
    Pm = np.linalg.matrix_power(P, m)
    assert np.all(Pm > 0), f"P^{m} has a non-positive entry:\n{Pm}"
```

```
P^1 =
[[0.33333333 0.66666667]
 [1.         0.        ]]

P^2 =
[[0.77777778 0.22222222]
 [0.33333333 0.66666667]]

P^3 =
[[0.48148148 0.51851852]
 [0.77777778 0.22222222]]

P^4 =
[[0.67901235 0.32098765]
 [0.48148148 0.51851852]]

P^5 =
[[0.5473251  0.4526749 ]
 [0.67901235 0.32098765]]
```

In [12]:

```
Pms = [np.linalg.matrix_power(P, m) for m in range(0,30)]
for m in range(20,30):
    Pm = Pms[m]
    Pm_1 = Pms[m-1]
    print(f"P^{m} =\n{Pm}\n; Diff =\n{Pm - Pm_1}\n")
```

```
P^20 =
[[0.60012029 0.39987971]
 [0.59981956 0.40018044]]
; Diff =
[[ 0.00030073 -0.00030073]
 [-0.00045109  0.00045109]]

P^21 =
[[0.59991981 0.40008019]
 [0.60012029 0.39987971]]
; Diff =
[[-0.00020049  0.00020049]
 [ 0.00030073 -0.00030073]]

P^22 =
[[0.60005346 0.39994654]
 [0.59991981 0.40008019]]
; Diff =
[[ 0.00013366 -0.00013366]
 [-0.00020049  0.00020049]]

P^23 =
[[0.59996436 0.40003564]
 [0.60005346 0.39994654]]
; Diff =
[[-8.91047881e-05  8.91047881e-05]
 [ 1.33657182e-04 -1.33657182e-04]]

P^24 =
[[0.60002376 0.39997624]
 [0.59996436 0.40003564]]
; Diff =
[[ 5.94031921e-05 -5.94031921e-05]
 [-8.91047881e-05  8.91047881e-05]]

P^25 =
[[0.59998416 0.40001584]
 [0.60002376 0.39997624]]
; Diff =
[[-3.96021280e-05  3.96021280e-05]
 [ 5.94031921e-05 -5.94031921e-05]]

P^26 =
[[0.60001056 0.39998944]
 [0.59998416 0.40001584]]
; Diff =
```

```
[[ 2.64014187e-05 -2.64014187e-05]
 [-3.96021280e-05  3.96021280e-05]]

P^27 =
[[0.59999296 0.40000704]
 [0.60001056 0.39998944]]
; Diff =
[[-1.76009458e-05  1.76009458e-05]
 [ 2.64014187e-05 -2.64014187e-05]]

P^28 =
[[0.60000469 0.39999531]
 [0.59999296 0.40000704]]
; Diff =
[[ 1.17339639e-05 -1.17339639e-05]
 [-1.76009458e-05  1.76009458e-05]]

P^29 =
[[0.59999687 0.40000313]
 [0.60000469 0.39999531]]
; Diff =
[[-7.82264258e-06  7.82264258e-06]
 [ 1.17339639e-05 -1.17339639e-05]]
```

Numerically, $lim_{n->\infty} P^n = [[0.6, 0.4], [0.6, 0.4]]$

To show irreducibility: a finite Markov chain is irreducible iff for every pair of states i,j ($i \neq j$), there exists some m such that $P^m_{i,j} > 0$. This indicates that there is transition between 2 (non-same) states. We have shown this above, where there is more than 1 positive non-diagonal element in each of P^m (for m ranging from 0 to 600).

The Markov chain is aperiodic because there exists $m \geq 1$ such that $(P^m)_{ii} > 0 \; for all states i. In particular, since (P)_{11} > 0 and (P)_{11} < 1, the chain can return to state 1 in one step, implying that the period of state 1 is equal to 1$. Because the chain is irreducible, all states share the same period, and therefore the chain is aperiodic.

p6

Exported 2/14/2026, 5:14:11 PM

6.1

In [25]:

```
import numpy as np
import scipy.io as sio
from scipy.integrate import quad

data = sio.loadmat('HW05_Problem6.mat')
y = data['observations'].flatten()
t = data['times'].flatten()

K = 20.5
sigma0 = 0.1
a, b = 1.2, 1.8

def z_model(t, q):
    omega = np.sqrt(K - q**2 / 4.0)
    return 2 * np.exp(-q * t / 2.0) * np.cos(omega * t)

def SS(q):
    return np.sum((y - z_model(t, q))**2)

def likelihood(q):
    return np.exp(-SS(q) / (2 * sigma0**2))

Z, _ = quad(likelihood, a, b)


q_val = 1.45
pi_val = likelihood(q_val) / Z

print("pi(q=1.45 | v) =", pi_val)

# ----------------------------
# Check that posterior integrates to 1
# ----------------------------
posterior_integral, _ = quad(lambda q: likelihood(q)/Z, a, b)
print("Integral of p(q|v) over [1.2,1.8] =", posterior_integral)
```

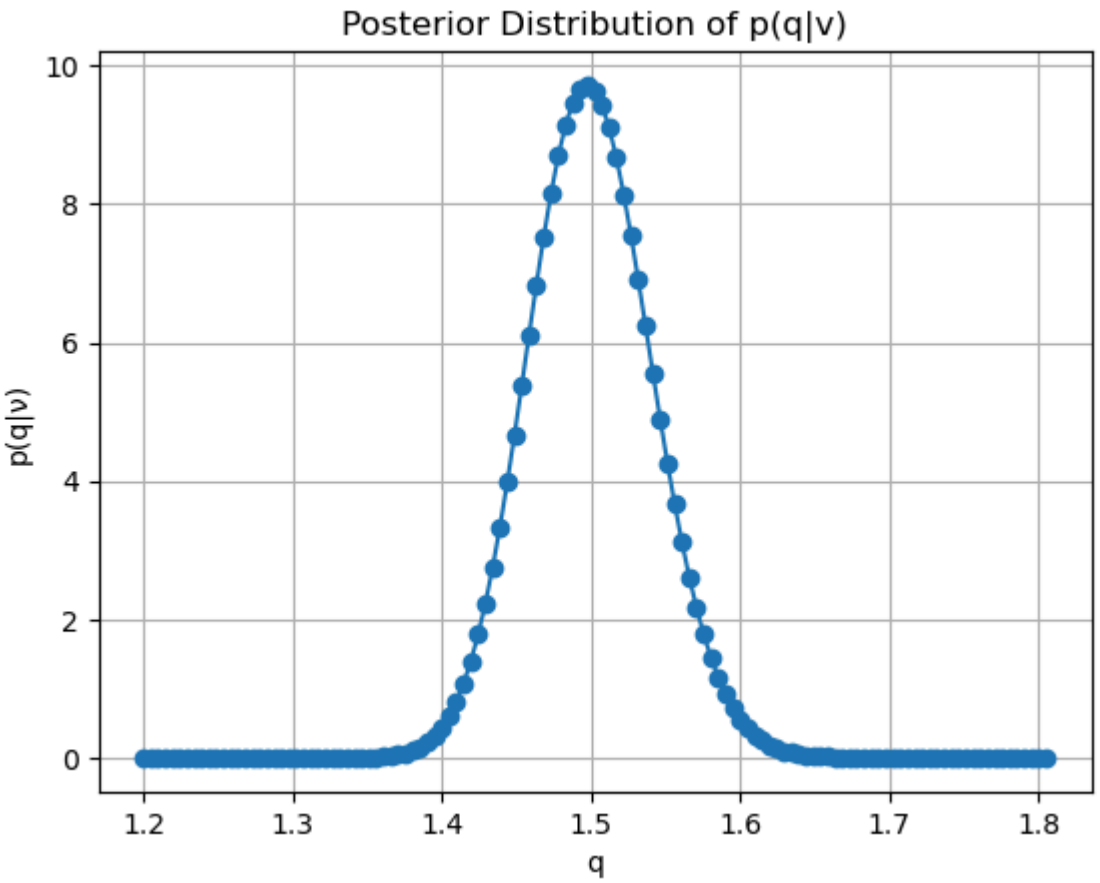
```
pi(q=1.45 | v) = 4.83920361390866
Integral of p(q|v) over [1.2,1.8] = 1.000000000000049
```

$\pi(q \mid v) \approx 4.83$

6.2

In [34]:

```
q_vals = np.linspace(1.2, 1.805, 125)
pi_qs = []
for q in q_vals:
    pi_q = likelihood(q) / Z
    # print(f"q = {q:.3f}, p(q|v) = {pi_q:.6f}")

    pi_qs += [pi_q]
import matplotlib.pyplot as plt
plt.plot(q_vals, pi_qs, 'o-')
plt.xlabel('q')
plt.ylabel('p(q|v)')
plt.title('Posterior Distribution of p(q|v)')
plt.grid()
plt.show()
```



6.3

In [41]:

```
qmap = q_vals[np.argmax(pi_qs)]
print("MAP pdf", pi_qs[np.argmax(pi_qs)])
print("MAP",qmap)
```

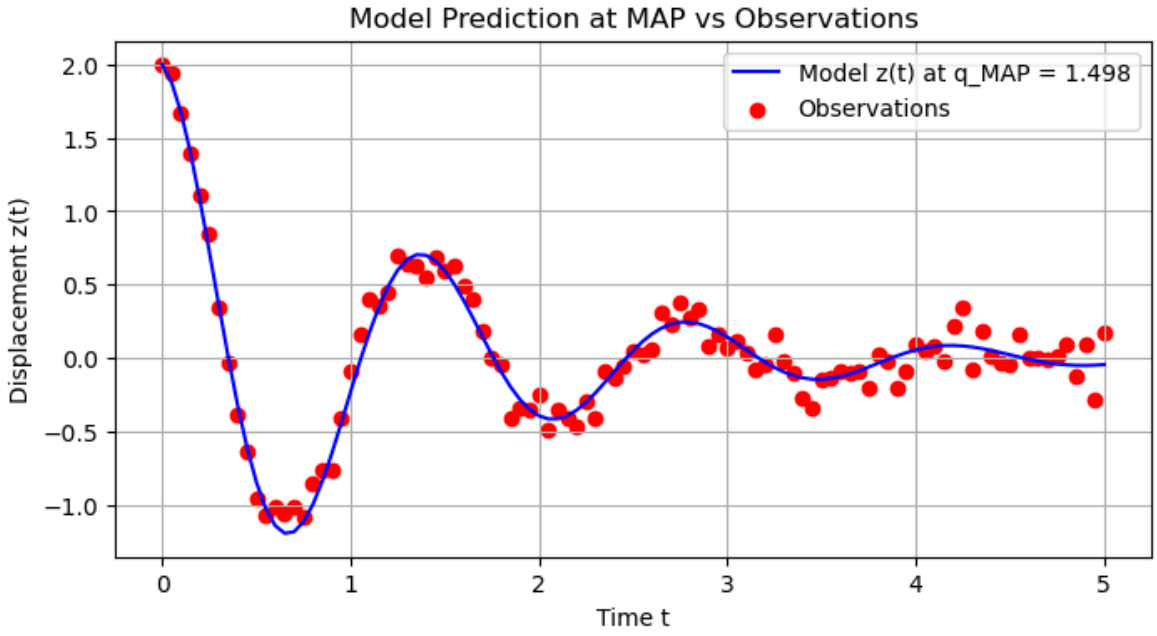
```
MAP pdf 9.723145265757058
MAP 1.4976209677419354
```

The MAP point is at $q_{MAP} \approx 1.497$ with probability density $p(q_{MAP} \mid v) \approx 9.72$.

6.4

In [42]:

```
z_map = z_model(t, qmap)

plt.figure(figsize=(8,4))
plt.plot(t, z_map, 'b-', label=f'Model z(t) at q_MAP = {qmap:.3f}')
plt.scatter(t, y, color='r', marker='o', label='Observations')
plt.xlabel('Time t')
plt.ylabel('Displacement z(t)')
plt.title('Model Prediction at MAP vs Observations')
plt.grid(True)
plt.legend()
plt.show()
```



6.5

The posterior distribution $\pi(q \mid v)$ provides a probabilistic estimate of the unknown damping coefficient C given the observed data. Using the discretized posterior evaluated over the range $[1.2, 1.805]$, the maximum a posteriori (MAP) estimate was found to be approximately $q_{\mathrm{MAP}} = 1.498$, which is extremely close to the true value $C_0 = 1.5$ used to generate the observations. This indicates that the data are informative and that the likelihood function, combined with the uniform prior, successfully identifies the region of high posterior probability around the true parameter. The posterior is relatively narrow, reflecting low uncertainty due to the modest observational noise ($\sigma_0 = 0.1$), and it is consistent with the true value within the numerical resolution of our discretized evaluation (e.g., the MAP is determined from a optimization over discrete points, rather than an optimization over the continuous parameter space).

p7

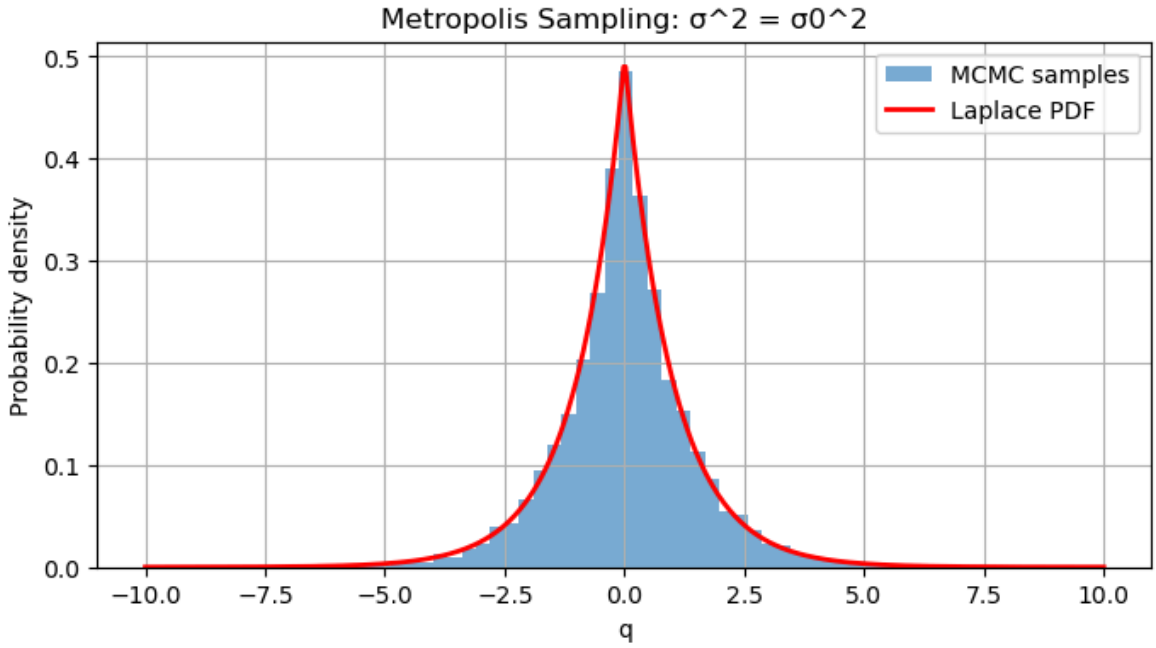
Exported 2/14/2026, 5:14:19 PM

7.1

In [ ]:

```
import numpy as np
import matplotlib.pyplot as plt
np.random.seed(0)

mu = 0
b = 1
sigma0_sq = 2 * b**2  # variance of Laplace
M = 10100
burn_in = 100

def p_laplace(q):
    """Laplace PDF."""
    return (1/(2*b)) * np.exp(-np.abs(q - mu)/b)

def metropolis_laplace(M, sigma_sq):
    samples = np.zeros(M)
    q_current = 0.0
    accepted = 0

    for k in range(M):
        q_star = np.random.normal(q_current, np.sqrt(sigma_sq))
        alpha = min(1, p_laplace(q_star) / p_laplace(q_current))
        if np.random.rand() < alpha:
            q_current = q_star
            accepted += 1
        samples[k] = q_current

    return samples[burn_in:], accepted / M

# ----------------------------
samples1, acc1 = metropolis_laplace(M, sigma0_sq)
print("Acceptance ratio (sigma^2 = sigma0^2):", acc1)

# Plot histogram
q_grid = np.linspace(-10, 10, 500)
plt.figure(figsize=(8,4))
plt.hist(samples1, bins=50, density=True, alpha=0.6, label='MCMC
samples')
plt.plot(q_grid, p_laplace(q_grid), 'r-', lw=2, label='Laplace PDF')
plt.xlabel('q')
plt.ylabel('Probability density')
plt.title('Metropolis Sampling: σ^2 = σ0^2')
plt.legend()
plt.grid(True)
plt.show()
```

```
Acceptance ratio (sigma^2 = sigma0^2): 0.620990099009901
```



In [ ]:

```
def check_pdf(samples, bins=100, tol=0.05):
    hist, edges = np.histogram(samples, bins=bins, density=True)
    bin_width = edges[1] - edges[0]
    integral = np.sum(hist * bin_width)
    is_pdf = np.abs(integral - 1.0) < tol
    return integral, is_pdf
integral, is_pdf = check_pdf(samples1, bins=50)
print(f"Numerical integral of histogram = {integral:.4f}, valid PDF?
{is_pdf}")
```

```
Numerical integral of histogram = 1.0000, is valid PDF? True
```

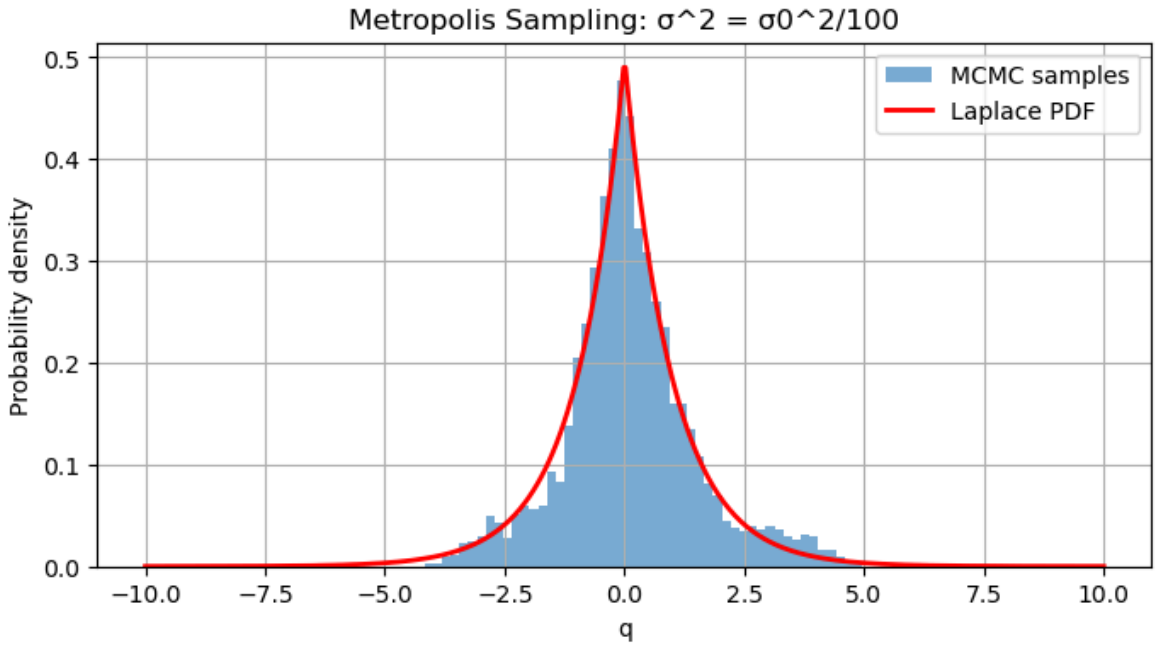
The acceptance ratio is approximately 0.621.

7.2

In [9]:

```
samples2, acc1 = metropolis_laplace(M, sigma0_sq/100)
print("Acceptance ratio (sigma^2 = sigma0^2):", acc1)

# Plot histogram
q_grid = np.linspace(-10, 10, 500)
plt.figure(figsize=(8,4))
plt.hist(samples2, bins=50, density=True, alpha=0.6, label='MCMC
samples')
plt.plot(q_grid, p_laplace(q_grid), 'r-', lw=2, label='Laplace PDF')
plt.xlabel('q')
plt.ylabel('Probability density')
plt.title('Metropolis Sampling: σ^2 = σ0^2/100')
plt.legend()
plt.grid(True)
plt.show()
```

```
Acceptance ratio (sigma^2 = sigma0^2): 0.9489108910891089
```

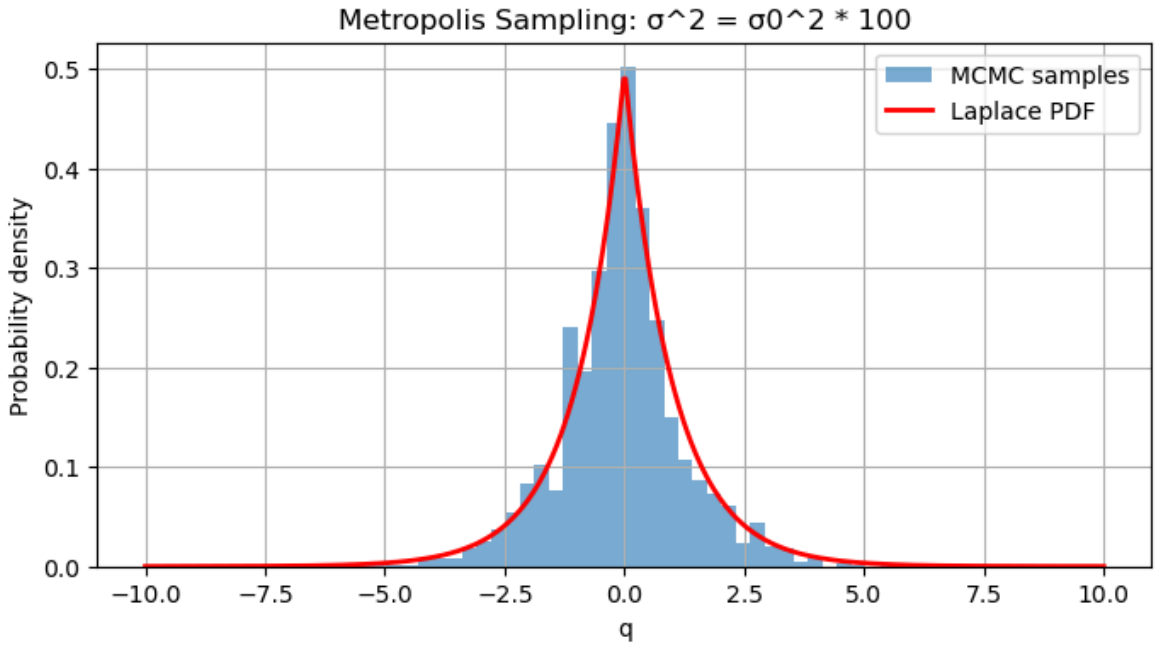


7.3

In [11]:

```
samples3, acc1 = metropolis_laplace(M, sigma0_sq*100)
print("Acceptance ratio (sigma^2 = sigma0^2):", acc1)

# Plot histogram
q_grid = np.linspace(-10, 10, 500)
plt.figure(figsize=(8,4))
plt.hist(samples3, bins=50, density=True, alpha=0.6, label='MCMC
samples')
plt.plot(q_grid, p_laplace(q_grid), 'r-', lw=2, label='Laplace PDF')
plt.xlabel('q')
plt.ylabel('Probability density')
plt.title('Metropolis Sampling: σ^2 = σ0^2 * 100')
plt.legend()
plt.grid(True)
plt.show()
```

```
Acceptance ratio (sigma^2 = sigma0^2): 0.10574257425742574
```



The proposal variance σ^2 has a significant effect on both the histogram of samples and the acceptance ratio in the Metropolis algorithm. When $\sigma^2 = \sigma_0^2 \,/\, 100$, the proposal steps are very small, so most proposed moves are accepted, resulting in an acceptance ratio close to 1. However, the chain explores the state space very slowly, leading to high autocorrelation and a histogram that may converge slowly to the true Laplace distribution (slow exploration). When $\sigma^2 = \sigma_0^2$, the proposal scale is well matched to the target distribution, producing a moderate acceptance ratio (0.621) and good mixing. In this case, the histogram closely matches the Laplace PDF. When $\sigma^2 = 100\sigma_0^2$, the proposal steps are very large, so most proposed moves land in low-density regions and are rejected. This results in a very low acceptance ratio and a chain that becomes "sticky," leading to poor exploration and a histogram that does not accurately approximate the target distribution. The histogram variance is, thus, much smaller when the variance of the proposal distribution is small (as few samples are accepted). These observations are more clearly seen in 7.5, at a smaller number of sample M.

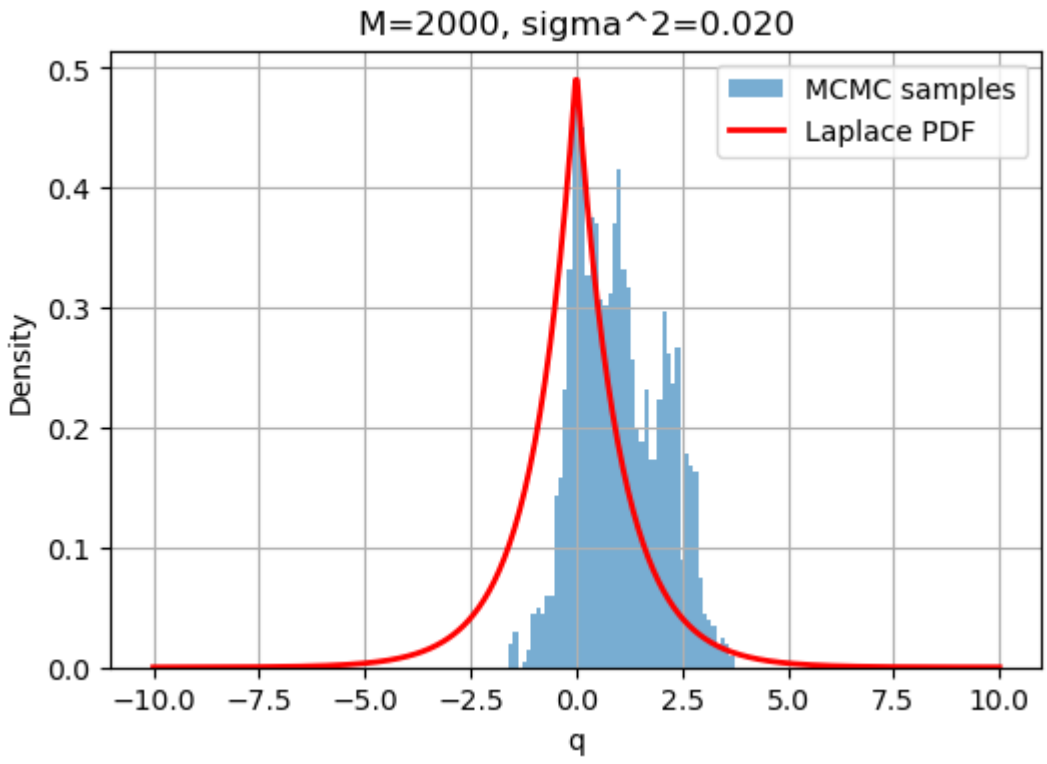
7.5

The number of samples M and the proposal variance σ^2 both strongly influence the quality of the resulting approximation to the Laplace distribution. Increasing M improves agreement with the true distribution by reducing Monte Carlo error and allowing the chain more time to explore the state space. However, if σ^2 is poorly chosen, we have inefficient sampling. If σ^2 is too small, the chain mixes slowly and requires a very large M to achieve good coverage. If σ^2 is too large, the chain experiences frequent rejections and remains stuck for long periods, also requiring a much larger M to approximate the target distribution well. Overall, good agreement with the Laplace distribution is achieved when σ^2 is tuned to produce a moderate acceptance ratio and M is large to ensure adequate mixing and convergence. Note that increasing M values in turn increases the computational cost of the algorithm.

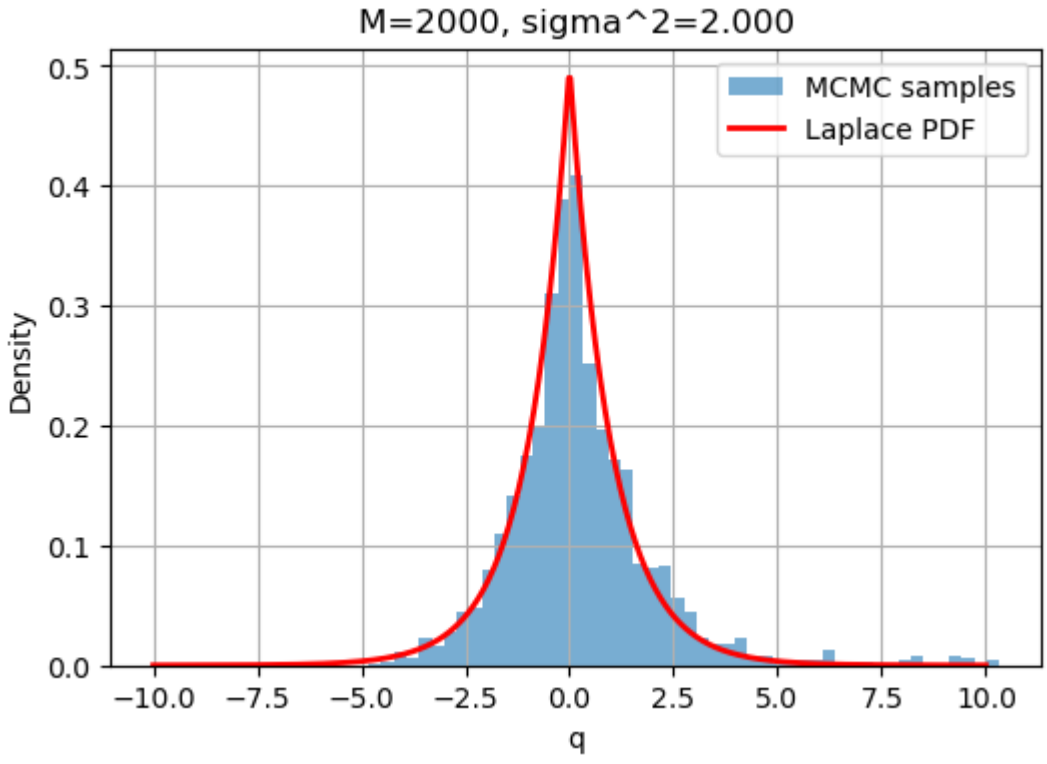
In [12]:

```
M_values = [2000, 10000, 50000]
sigma_values = [sigma0_sq/100, sigma0_sq, 100*sigma0_sq]

for M in M_values:
    for sigma_sq in sigma_values:
        samples, acc_ratio = metropolis_laplace(M, sigma_sq)

        print(f"M = {M}, sigma^2 = {sigma_sq:.3f}, acceptance ratio =
{acc_ratio:.3f}")

        # Plot
        q_grid = np.linspace(-10, 10, 500)

        plt.figure(figsize=(6,4))
        plt.hist(samples, bins=50, density=True, alpha=0.6,
label="MCMC samples")
        plt.plot(q_grid, p_laplace(q_grid), 'r-', lw=2, label="Laplace
PDF")
        plt.title(f"M={M}, sigma^2={sigma_sq:.3f}")
        plt.xlabel("q")
        plt.ylabel("Density")
        plt.legend()
        plt.grid(True)
        plt.show()
```

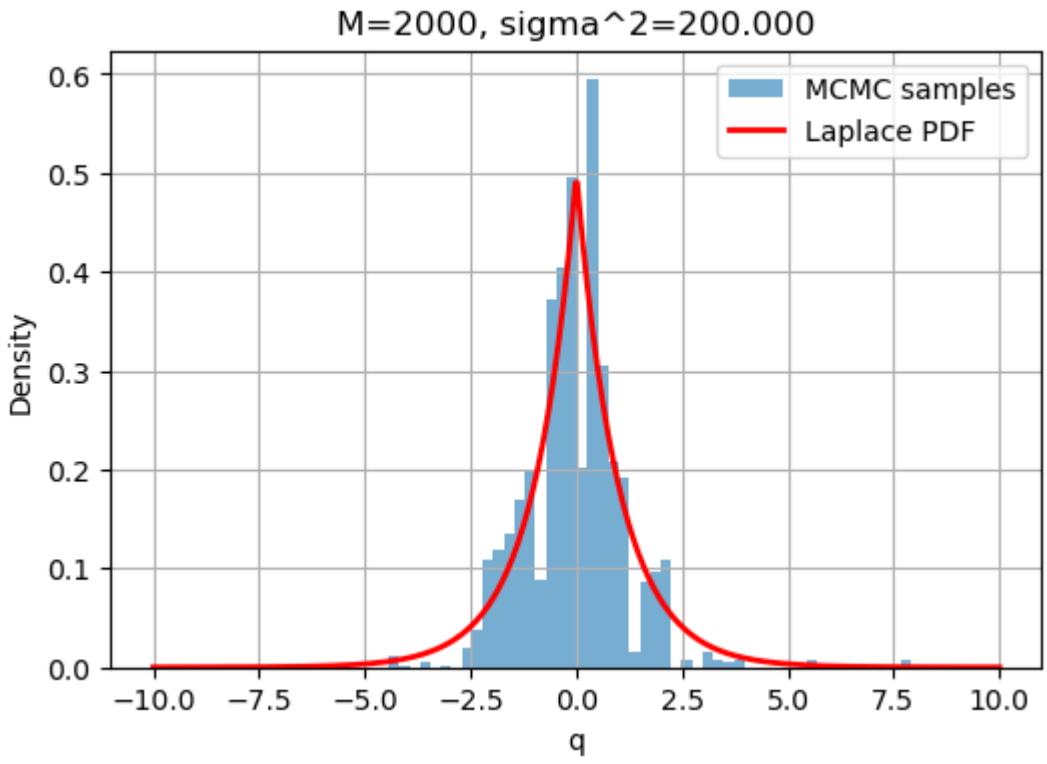
```
M = 2000, sigma^2 = 0.020, acceptance ratio = 0.947
```



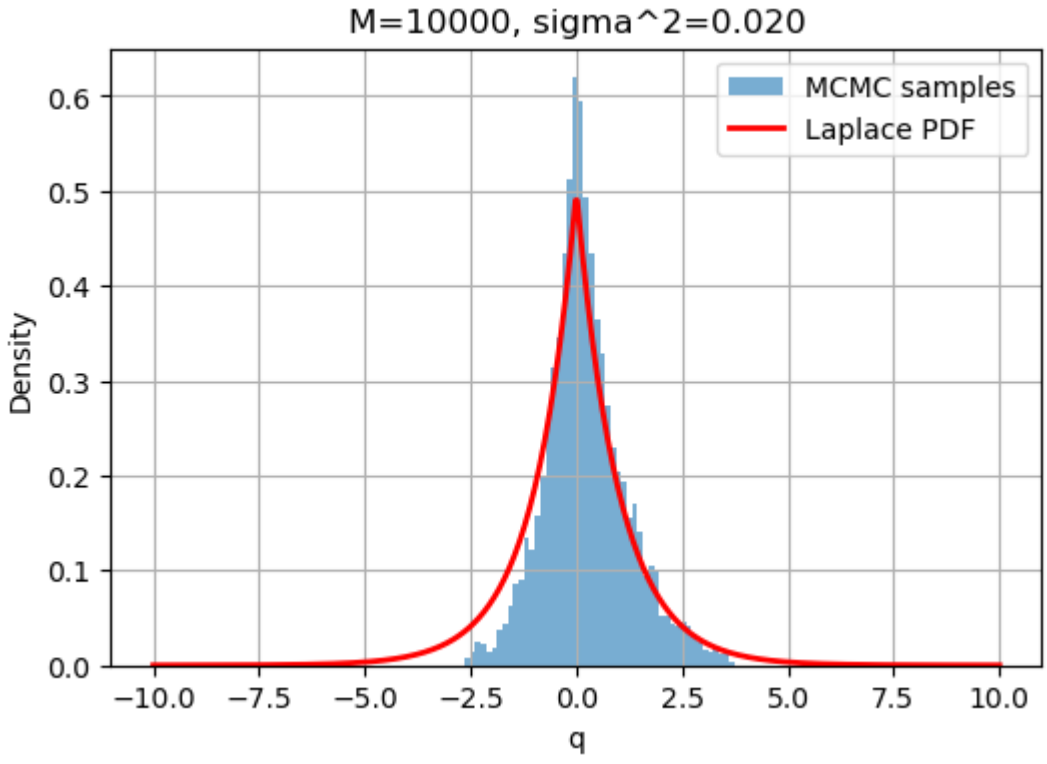

```
M = 2000, sigma^2 = 2.000, acceptance ratio = 0.627
```



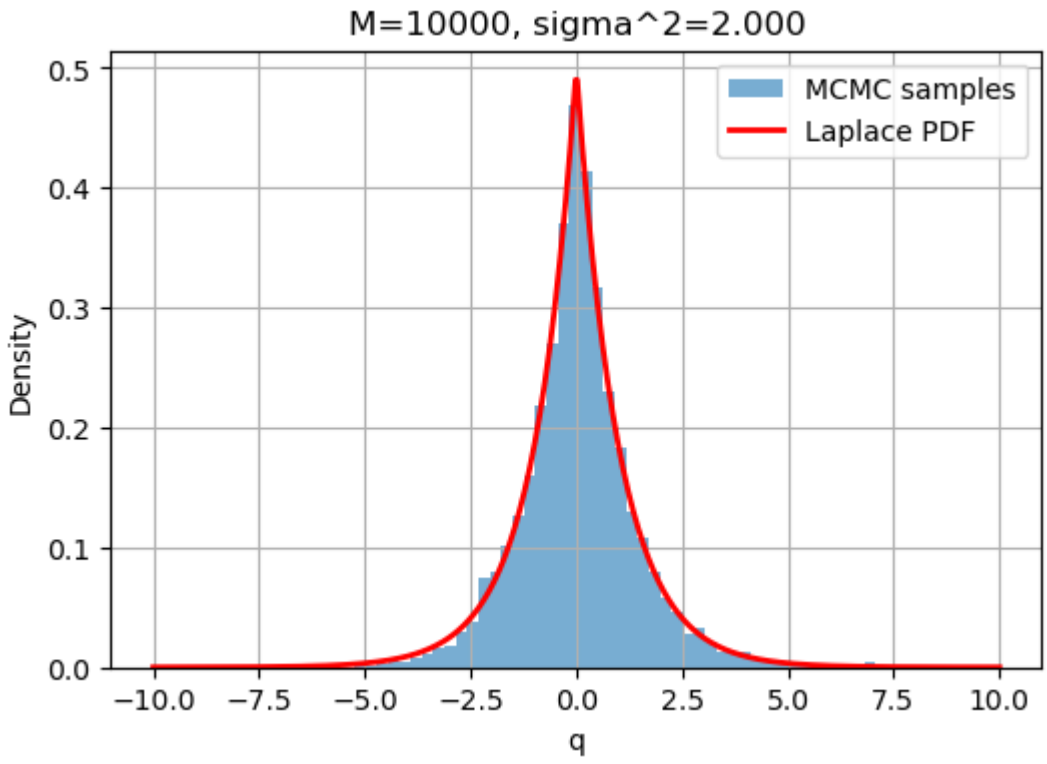

```
M = 2000, sigma^2 = 200.000, acceptance ratio = 0.103
```



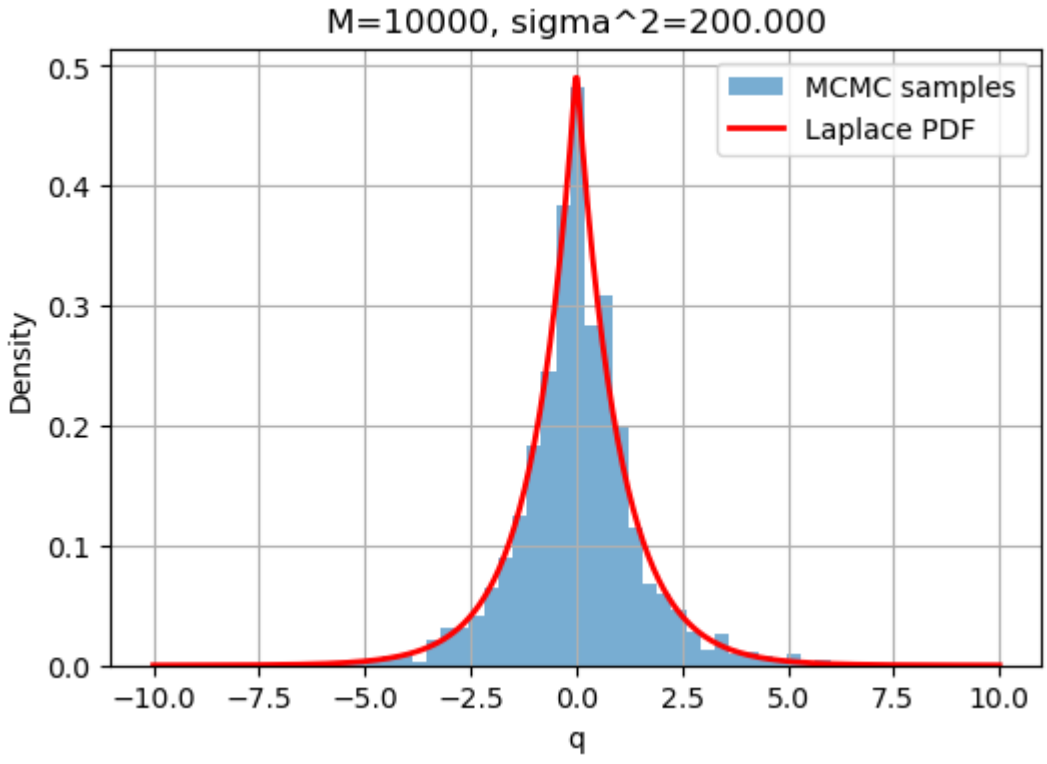

```
M = 10000, sigma^2 = 0.020, acceptance ratio = 0.944
```



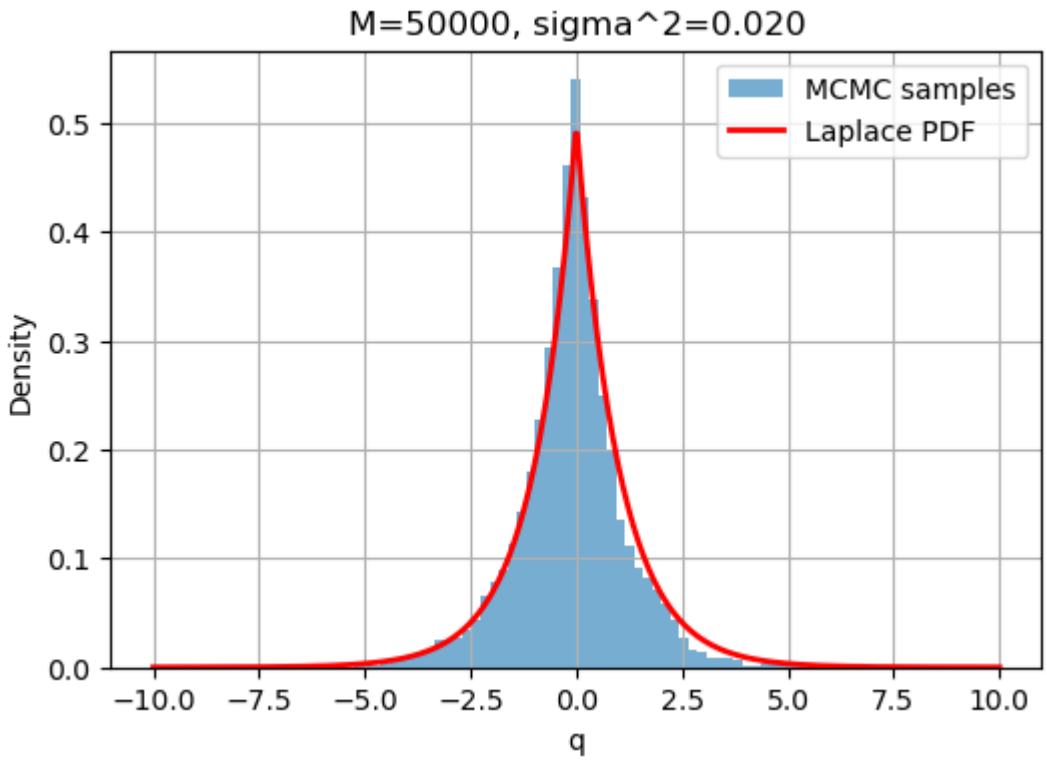

```
M = 10000, sigma^2 = 2.000, acceptance ratio = 0.623
```



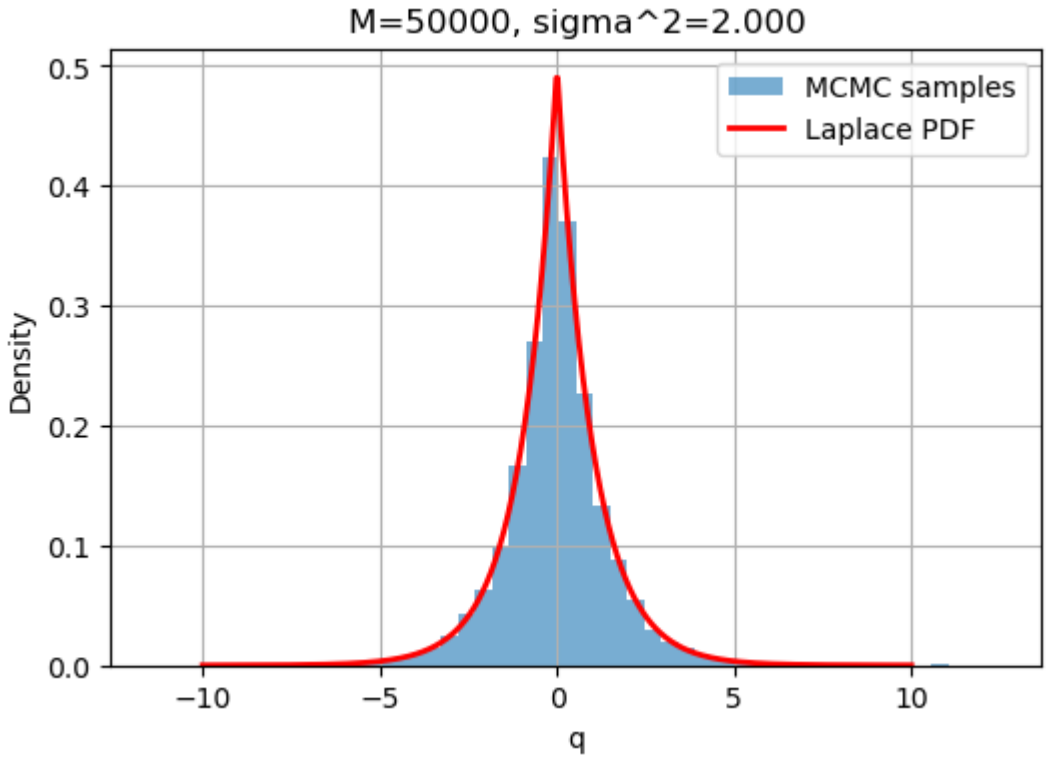

```
M = 10000, sigma^2 = 200.000, acceptance ratio = 0.110
```




```
M = 50000, sigma^2 = 0.020, acceptance ratio = 0.946
```




```
M = 50000, sigma^2 = 2.000, acceptance ratio = 0.612
```




```
M = 50000, sigma^2 = 200.000, acceptance ratio = 0.107
```